

Laporan Optimasi Multi-Objektif Flow Shop Scheduling Problem Menggunakan Walrus Optimization Algorithm (WaOA)



Kelompok 5

Bryan Ricaldi Chandra — 6182301013
Vince Farrel Natanael — 6182301024
Robert Saputra — 6182301080
Bima Rahmadani Arifandi — 6182301094

25 Juni 2026

1 Pendahuluan

Optimalisasi alokasi sumber daya dan penjadwalan tugas (*task scheduling*) merupakan salah satu tantangan fundamental dalam pengembangan sistem komputasi performa tinggi (*high-performance computing*), arsitektur multiprosesor, sistem terdistribusi, hingga *cloud computing*. Salah satu model teoritis yang sangat relevan untuk merepresentasikan antrean eksekusi proses ini adalah *Flow Shop Scheduling Problem (FSSP)*. Pada rumpun informatika, FSSP memodelkan sekumpulan program atau instruksi (*jobs*) yang harus dieksekusi melalui serangkaian tahapan pemrosesan atau perangkat pemroses (*machines*) dengan urutan interaksi alur kerja (*pipeline*) yang seragam. Secara komputasi, permasalahan ini masuk dalam kategori komputasi kombinatorial *NP-hard*, yang berarti pencarian solusi optimal global secara *exhaustive search* tidak layak dijalankan seiring dengan peningkatan skala data instruksi.

Pada implementasi arsitektur perangkat lunak dan sistem operasi modern, seorang insinyur komputer tidak hanya dihadapkan pada satu parameter performa saja. Penjadwalan proses umumnya bersifat multi-objektif (*Multi-Objective*). Di satu sisi, sistem dituntut untuk meminimalkan total waktu eksekusi keseluruhan tugas (*Makespan*) demi meningkatkan *throughput* dan utilitas prosesor. Di sisi lain, sistem juga harus meminimalkan total waktu tanggap atau alir (*Total Flow Time*) guna meminimalkan *latency* individual dan mencegah terjadinya *starvation* pada antrean instruksi. Kompleksitas trade-off antar fungsi tujuan yang saling bertentangan ini menuntut ditemukannya sekumpulan solusi kompromi optimal yang menyusun *Pareto-optimal front*.

Guna memecahkan dilema optimasi tersebut, algoritma metaheuristik berbasis populasi menjadi pendekatan yang dominan dalam riset informatika karena efisiensinya dalam mengeksplorasi ruang pencarian (*search space*) yang luas. Laporan ini mengeksplorasi dan mengadaptasi algoritma metaheuristik baru bernama *Walrus Optimization Algorithm (WaOA)* yang diusulkan oleh Trojovský & Dehghani (2022). Algoritma ini meniru pola perilaku biologis cerdas dari mamalia walrus. Melalui pemetaan matematis dari ruang pencarian kontinu ke ruang diskret serta integrasi konsep *Pareto dominance*, algoritma WaOA dikembangkan menjadi algoritma multi-objektif (MO-WaOA) untuk menghasilkan skema penjadwalan eksekusi tugas yang optimal pada problem MO-FSSP.

2 Deskripsi Permasalahan MO-FSSP

Secara formal, Multi-Objective Flow Shop Scheduling Problem (MO-FSSP) dengan konfigurasi n buah tugas (*jobs*) dan m tahapan pemroses (*machines*) dapat didefinisikan sebagai berikut:

Diberikan himpunan tugas $J = \{1, 2, \dots, n\}$ dan himpunan pemroses $M = \{1, 2, \dots, m\}$. Setiap tugas $i \in J$ wajib diproses pada pemroses $j \in M$ dengan durasi waktu komputasi deterministik $p_{i,j}$. Alur eksekusi seluruh tugas bersifat *non-preemptive*, yang berarti suatu proses yang sedang berjalan di sebuah core tidak dapat diinterupsi sebelum selesai, dan setiap core hanya mampu melayani satu tugas pada satu satuan waktu.

Tujuan utama dari masalah ini adalah menentukan urutan permutasi eksekusi tugas terbaik $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ dari ruang permutasi yang mungkin. Waktu penyelesaian tugas pada urutan ke- i di pemroses ke- j , dinotasikan sebagai $C(\pi_i, j)$, dihitung secara rekursif melalui pemodelan matematika di bawah ini:

$$C(\pi_1, 1) = p_{\pi_1, 1} \quad (1)$$

$$C(\pi_i, 1) = C(\pi_{i-1}, 1) + p_{\pi_i, 1} \quad \text{untuk } i = 2, \dots, n \quad (2)$$

$$C(\pi_1, j) = C(\pi_1, j - 1) + p_{\pi_1, j} \quad \text{untuk } j = 2, \dots, m \quad (3)$$

$$C(\pi_i, j) = \max(C(\pi_{i-1}, j), C(\pi_i, j - 1)) + p_{\pi_i, j} \quad \text{untuk } i = 2, \dots, n; j = 2, \dots, m \quad (4)$$

Dua fungsi objektif informatika yang dioptimalkan secara simultan dalam laporan ini adalah:

1. **Minimasi Makespan (C_{max}):** Meminimalkan total waktu penyelesaian komputasi pada pemroses terakhir demi efisiensi *hardware*.

$$f_1(\pi) = C_{max}(\pi) = C(\pi_n, m) \quad (5)$$

2. **Minimasi Total Flow Time (TFT):** Meminimalkan akumulasi waktu tunggu dan waktu proses seluruh tugas di dalam sistem untuk menekan rata-rata *turnaround time*.

$$f_2(\pi) = \sum_{i=1}^n C(\pi_i, m) \quad (6)$$

Formulasi optimasi multi-objektif ini dituliskan sebagai:

$$\min F(\pi) = \{f_1(\pi), f_2(\pi)\} \quad (7)$$

3 Penjelasan Algoritma Metaheuristik Walrus Optimization Algorithm (WaOA)

Walrus Optimization Algorithm (WaOA) memanfaatkan kecerdasan buatan berbasis kawanan (*swarm intelligence*) yang memodelkan interaksi sosial populasi walrus ke dalam tiga fase utama penjelajahan ruang pencarian:

3.1 Inisialisasi Populasi

Pada awal komputasi, representasi posisi agen pencari (walrus) diplot dalam matriks kontinu berukuran $N \times n$:

$$X = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_i \\ \vdots \\ X_N \end{bmatrix}_{N \times n} = \begin{bmatrix} x_{1,1} & \dots & x_{1,j} & \dots & x_{1,n} \\ x_{2,1} & \dots & x_{2,j} & \dots & x_{2,n} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{i,1} & \dots & x_{i,j} & \dots & x_{i,n} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{N,1} & \dots & x_{N,j} & \dots & x_{N,n} \end{bmatrix}_{N \times n} \quad (8)$$

Di mana N menyatakan ukuran populasi walrus dan n melambangkan dimensi variabel keputusan (jumlah tugas).

3.2 Fase 1: Strategi Mencari Makan / Feeding Strategy (Eksplorasi Global)

Aktivitas pencarian makan dipandu oleh walrus dengan gading terkuat (*Strongest Walrus, SW*). Perilaku bimbingan kelompok ini memodelkan lompatan eksplorasi global algoritma. Pembaruan posisi dimensi ke- j dari agen ke- i dirumuskan sebagai berikut:

$$x_{i,j}^{P_1} = x_{i,j} + rand \cdot (SW_j - I \cdot x_{i,j}) \quad (9)$$

Di mana $rand$ bernilai acak acak $[0, 1]$, dan $I \in \{1, 2\}$ berperan sebagai konstanta penentu bobot penjelajahan.

3.3 Fase 2: Migrasi / Migration (Eksplorasi)

Fase migrasi memodelkan diversifikasi solusi dengan bergerak menuju posisi agen lain (X_k) yang dipilih acak dari populasi aktif:

$$x_{i,j}^{P_2} = \begin{cases} x_{i,j} + rand \cdot (x_{k,j} - I \cdot x_{i,j}), & \text{jika } F_k \text{ mendominasi } F_i \\ x_{i,j} + rand \cdot (x_{i,j} - x_{k,j}), & \text{lainnya} \end{cases} \quad (10)$$

3.4 Fase 3: Melarikan Diri dari Predator / Escaping Predators (Eksplorasi Lokal)

Guna menghindari predator, walrus melakukan manuver dalam radius ketetanggaan lokal (*neighborhood search*) yang menyusut secara adaptif seiring bertambahnya iterasi generasi t :

$$lb_{local,j}^t = \frac{lb_j}{t}, \quad ub_{local,j}^t = \frac{ub_j}{t} \quad (11)$$

$$x_{i,j}^{P_3} = x_{i,j} + (lb_{local,j}^t + (ub_{local,j}^t - rand \cdot lb_{local,j}^t)) \quad (12)$$

4 Representasi Solusi dan Mekanisme Pembangkitan Solusi

4.1 Aturan Pemetaan Smallest Position Value (SPV)

Mengingat WaOA asli bekerja pada domain koordinat kontinu sedangkan penjadwalan FSSP adalah masalah diskret kombinatorial, algoritma diintegrasikan dengan metode *Smallest Position Value* (SPV). Melalui SPV, nilai-nilai kontinu dalam vektor posisi diurutkan secara *ascending order*, dan indeks asal elemen sebelum pengurutan akan diekstrak menjadi deret permutasi tugas diskret π .

Contoh Ilustrasi: Bila suatu rancangan posisi kontinu walrus X_i untuk 4 tugas ($n = 4$) bernilai:

$$X_i = [1.25, -0.45, 2.10, 0.15]$$

Urutan elemen dari nilai terkecil ke terbesar beserta indeks asalnya:

- $-0.45 \rightarrow$ Berada di Indeks Asal 2
- $0.15 \rightarrow$ Berada di Indeks Asal 4
- $1.25 \rightarrow$ Berada di Indeks Asal 1
- $2.10 \rightarrow$ Berada di Indeks Asal 3

Maka, hasil konversi menghasilkan urutan susunan eksekusi tugas diskret: $\pi = (2, 4, 1, 3)$.

4.2 Adaptasi Arsitektur Multi-Objektif (MO-WaOA)

1. **Pareto Dominance:** Solusi π_A dikatakan mendominasi π_B ($\pi_A \prec \pi_B$) jika kualitas solusi π_A tidak lebih buruk pada semua objektif (f_1 dan f_2) serta mutlak lebih unggul pada minimal satu objektif.
2. **External Archive Control:** Repositori eksternal (*External Archive*) dialokasikan khusus untuk menyimpan kandidat solusi yang bersifat non-dominated sepanjang iterasi berlangsung dengan kapasitas maksimal yang ditentukan (misalnya 100 solusi). Untuk menjaga efisiensi komputasi dan membatasi ukuran repositori, mekanisme pemangkasan berbasis

urutan masuk (*First-In, First-Out / FIFO*) diterapkan, di mana solusi tertua di dalam arsip akan dieliminasi apabila solusi non-dominated baru masuk saat kapasitas arsip telah penuh.

3. **Dynamic Leader Assignment:** Penentuan pemimpin kelompok (*SW*) dilakukan secara acak merata (*Uniform Random Selection*) dengan mengambil salah satu anggota dari External Archive. Melalui mekanisme ini, setiap solusi non-dominated yang berhasil bertahan di dalam arsip memiliki probabilitas atau peluang yang sama besar untuk terpilih sebagai pemandu arah pergerakan populasi walrus pada Fase Strategi Pemberian Makan (*Feeding Strategy*).

5 Implementasi Solver

Implementasi solver ini dibagi menjadi dua komponen inti, yaitu manajemen penyimpanan solusi Pareto non-dominated (`ParetoArchive.java`) dan logika eksekusi siklus optimasi swarm WaOA (`WaOA.java`).

5.1 Manajemen Solusi Non-Dominated (Pareto Archive)

Dalam optimasi multi-objektif berbasis Pareto dominance, tidak ditemukan solusi tunggal yang mutlak terbaik. Oleh karena itu, diperlukan sebuah repositori dinamis eksternal yang berfungsi sebagai memori jangka panjang untuk menyimpan seluruh kandidat solusi yang saling tidak mendominasi (*non-dominated solutions*) selama iterasi berlangsung.

Kelas `ParetoArchive` mengimplementasikan mekanisme *elitism* dengan batasan ukuran maksimum (`maxSize`) sebesar 100 agen. Pembatasan ini bertujuan untuk menjaga efisiensi komputasi dari proses penyaringan dominansi yang bernilai kompleksitas kuadratik $\mathcal{O}(A^2)$, di mana A adalah jumlah elemen di dalam arsip. Proses pembaruan (*update*) arsip dilakukan dengan aturan kontraktual berikut:

1. Setiap solusi baru ($\vec{x}_{new}$) yang dihasilkan oleh fase mutasi WaOA akan diuji terhadap seluruh anggota arsip aktif.
2. Jika terdapat minimal satu anggota arsip yang mendominasi $\vec{x}_{new}$, maka solusi baru tersebut langsung ditolak (*discarded*).
3. Jika $\vec{x}_{new}$ mendominasi beberapa anggota arsip lama, maka seluruh elemen arsip yang didominasi tersebut akan dihapus secara permanen dari memori, dan $\vec{x}_{new}$ dimasukkan ke dalam arsip.
4. Jika kapasitas arsip melampaui `maxSize`, solusi terluar pada elemen ekor (*tail*) akan dipotong demi menjaga diversitas distribusi spasial *Pareto front*.

Implementasi komponen manajemen berkas arsip ini dituliskan pada Kode Program 1.

```

1 package waoa;
2
3 import java.util.ArrayList;
4 import java.util.Arrays;
5 import java.util.List;
6
7 public class ParetoArchive {
8     private final List<Walrus> archive;
9     private final int maxSize;
10
11
12     public ParetoArchive(int maxSize) {
```

```

13         this.archive = new ArrayList<>();
14         this.maxSize = maxSize;
15     }
16
17     public synchronized void update(Walrus candidate) {
18         Walrus copy = candidate.cloneWalrus();
19         List<Walrus> toRemove = new ArrayList<>();
20
21         for (Walrus archived : archive) {
22             boolean isSequenceDuplicate = Arrays.equals(archived.jobSequence,
23 copy.jobSequence);
24             if (isSequenceDuplicate) {
25                 return;
26             }
27             if (archived.dominates(copy)) {
28                 return; // Jika kandidat didominasi oleh isi arsip, tolak
29             }
30             if (copy.dominates(archived)) {
31                 toRemove.add(archived); // Jika kandidat mendominasi isi arsip,
32 tandai hapus
33             }
34         }
35
36         archive.removeAll(toRemove);
37         archive.add(copy);
38
39         if (archive.size() > maxSize) {
40             archive.remove(archive.size() - 1);
41         }
42     }
43
44     public synchronized List<Walrus> getArchive() {
45         return new ArrayList<>(archive);
46     }
47 }

```

Listing 1: Implementasi Kelas Pareto Archive

5.2 Arsitektur Algoritma Eksekusi WaOA

Kelas utama solver `WaOA.java` bertanggung jawab mengendalikan metamorfosis posisi kawanan walrus melewati ruang pencarian kontinu berdimensi n dengan batasan bawah ($LB = -10.0$) dan batasan atas ($UB = 10.0$). Metode `solve()` mensimulasikan evolusi populasi secara sekuensial melalui tiga fase biologis utama:

- **Eksekusi Eksplorasi Fase 1:** Menentukan pemimpin sementara (*Strongest Walrus, SW*) yang dipilih secara acak dari solusi-solusi terbaik di dalam `ParetoArchive`. Pemilihan acak dari arsip ini memberikan tekanan seleksi (*selection pressure*) yang seimbang agar pergerakan populasi tidak terjebak pada satu titik lokal saja (*premature convergence*).
- **Eksekusi Diversifikasi Fase 2:** Membandingkan dominansi antara walrus saat ini dengan walrus lain yang dipilih secara acak (X_k). Jika X_k terbukti mendominasi walrus saat ini, pergerakan diarahkan mendekati X_k . Jika tidak, walrus menjauh dari koordinat X_k untuk memperluas cakupan wilayah pencarian.
- **Eksekusi Eksploitasi Fase 3:** Mengurangi parameter jangkauan pencarian lokal secara berbanding terbalik terhadap indeks generasi aktif (t). Persamaan $lb_{local} = LB/t$ memaksa populasi melakukan penyisiran halus (*fine-tuning*) di sekitar area optimal pada iterasi-iterasi akhir.

Struktur parsial dan inti siklus komputasi dari kelas WaOA ditunjukkan pada Kode Program

2.

```
1 package waoa;
2
3 import model.ProblemInstance;
4 import java.util.List;
5 import java.util.Random;
6
7 public class WaOA {
8     private final ProblemInstance problem;
9     private final int popSize;
10    private final int maxIterations;
11    private final Random rand;
12
13    private final double LB = -10.0;
14    private final double UB = 10.0;
15
16    public WaOA(ProblemInstance problem, int popSize, int maxIterations) {
17        this.problem = problem;
18        this.popSize = popSize;
19        this.maxIterations = maxIterations;
20        this.rand = new Random();
21    }
22
23    public List<Walrus> solve() {
24        int numJobs = problem.getNumJobs();
25        Population population = new Population(popSize, numJobs);
26        population.initializeRandomly(rand, LB, UB);
27
28        ParetoArchive archive = new ParetoArchive(100);
29
30        // Evaluasi objektif awal untuk seluruh populasi
31        for (Walrus walrus : population.getMembers()) {
32            walrus.objectives = evaluateObjectives(walrus.jobSequence);
33            archive.update(walrus);
34        }
35
36        // Loop Iterasi Utama WaOA
37        for (int t = 1; t <= maxIterations; t++) {
38            List<Walrus> members = population.getMembers();
39
40            for (int i = 0; i < popSize; i++) {
41                Walrus current = members.get(i);
42
43                // -----
44                // PHASE 1: Feeding Strategy (Exploration)
45                // -----
46                List<Walrus> leaders = archive.getArchive();
47                Walrus sw = leaders.get(rand.nextInt(leaders.size())); //
48                Strongest Walrus dari arsip
49
50                Walrus p1 = new Walrus(numJobs);
51                for (int j = 0; j < numJobs; j++) {
52                    int I = rand.nextInt(2) + 1;
53                    p1.position[j] = current.position[j] + rand.nextDouble() * (
54                        sw.position[j] - I * current.position[j]);
55                }
56                p1.jobSequence = population.convertPositionToSequence(p1.
57                    position);
58                p1.objectives = evaluateObjectives(p1.jobSequence);
59
60                if (!current.dominates(p1)) {
61                    members.set(i, p1);
62                }
63            }
64        }
65    }
66}
```

```

59         current = p1;
60         archive.update(p1);
61     }
62
63     // -----
64     // PHASE 2: Migration
65     // -----
66     int k;
67     do { k = rand.nextInt(popSize); } while (k == i);
68     Walrus target = members.get(k);
69
70     Walrus p2 = new Walrus(numJobs);
71     boolean targetDominates = target.dominates(current);
72
73     for (int j = 0; j < numJobs; j++) {
74         if (targetDominates) {
75             int I = rand.nextInt(2) + 1;
76             p2.position[j] = current.position[j] + rand.nextDouble()
* (target.position[j] - I * current.position[j]);
77         } else {
78             p2.position[j] = current.position[j] + rand.nextDouble()
* (current.position[j] - target.position[j]);
79         }
80     }
81     p2.jobSequence = population.convertPositionToSequence(p2.
position);
82     p2.objectives = evaluateObjectives(p2.jobSequence);
83
84     if (!current.dominates(p2)) {
85         members.set(i, p2);
86         current = p2;
87         archive.update(p2);
88     }
89
90     // -----
91     // PHASE 3: Escaping and Fighting Against Predators (
Exploitation)
92     // -----
93     Walrus p3 = new Walrus(numJobs);
94     double lbLocal = LB / t;
95     double ubLocal = UB / t;
96
97     for (int j = 0; j < numJobs; j++) {
98         p3.position[j] = current.position[j] + (lbLocal + (ubLocal -
rand.nextDouble() * lbLocal));
99     }
100    p3.jobSequence = population.convertPositionToSequence(p3.
position);
101    p3.objectives = evaluateObjectives(p3.jobSequence);
102
103    if (!current.dominates(p3)) {
104        members.set(i, p3);
105        archive.update(p3);
106    }
107    }
108    }
109    return archive.getArchive();
110 }
111
112 private double[] evaluateObjectives(int[] sequence) {
113     int[][] pTime = problem.getProcessingTimes();
114     int n = problem.getNumJobs();
115     int m = problem.getNumMachines();

```

```

116     int[][] C = new int[n][m];
117
118     C[0][0] = pTime[sequence[0]][0];
119     for (int j = 1; j < m; j++) {
120         C[0][j] = C[0][j-1] + pTime[sequence[0]][j];
121     }
122
123     for (int i = 1; i < n; i++) {
124         C[i][0] = C[i-1][0] + pTime[sequence[i]][0];
125         for (int j = 1; j < m; j++) {
126             C[i][j] = Math.max(C[i-1][j], C[i][j-1]) + pTime[sequence[i]][j];
127         }
128     }
129
130     double makespan = C[n-1][m-1];
131     double totalFlowTime = 0;
132     for (int i = 0; i < n; i++) {
133         totalFlowTime += C[i][m-1];
134     }
135
136     return new double[]{makespan, totalFlowTime};
137 }
138 }

```

Listing 2: Struktur Inti Algoritma pada Kelas WaOA

Setiap perubahan posisi baru hasil mutasi pada Kode Program 2 diverifikasi menggunakan percabangan `if (!current.dominates(p))`. Logika ini mematuhi hukum Pareto, di mana koordinat baru walrus akan diterima sebagai posisi *survival* yang baru jika posisinya terbukti lebih baik atau minimal setara (tidak didominasi) oleh posisi lama agen tersebut.

6 Desain Eksperimen

Bagian ini menjelaskan rancangan eksperimen yang dilakukan untuk menguji performa algoritma *Multi-Objective Walrus Optimization Algorithm* (MOWaOA) dalam menyelesaikan *Flow Shop Scheduling Problem* (FSSP). Desain eksperimen ini mencakup karakteristik data uji (*benchmark*), konfigurasi parameter algoritma, serta metrik yang digunakan untuk mengevaluasi kualitas solusi.

6.1 Data Uji (*Benchmark Instances*)

Eksperimen ini menggunakan data uji standar internasional yang dikembangkan oleh Taillard (1993) untuk kasus *Flow Shop Scheduling*. Penelitian ini menggunakan lima variasi berkas uji coba dengan kombinasi jumlah *jobs* (n) dan jumlah mesin (m) yang berbeda guna merepresentasikan berbagai tingkat skala dan kompleksitas ruang pencarian solusi ($n!$). Karakteristik dari kelima data uji tersebut dirangkum dalam Tabel 1.

Tabel 1: Karakteristik Dokumen Data Uji Taillard Benchmark yang Digunakan

No.	Nama Berkas	Jumlah Kerja (<i>Jobs</i>)	Jumlah Mesin (<i>Machines</i>)	Skala Kompleksitas
1	tai100.5	100	5	Besar (Fokus pada <i>Jobs</i>)
2	tai50.10	50	10	Menengah
3	tai100.10	100	10	Besar
4	tai50.20	50	20	Menengah-Besar
5	tai100.20	100	20	Sangat Besar (<i>High-Dimensional</i>)

Pemilihan kelima jenis data uji di atas bertujuan untuk mengevaluasi ketangguhan (*scalability*) algoritma MOWaOA. Variasi jumlah *jobs* hingga mencapai 100 unit akan menguji efisiensi algoritma dalam menghindari jebakan lokal optimal (*local optima*) pada ruang pencarian yang sangat luas, sedangkan variasi jumlah mesin hingga 20 unit akan menguji kemampuan algoritma dalam menangani akumulasi waktu tunggu (*idle time*) pada setiap stasiun kerja.

6.2 Skenario Pengujian Parameter (*Parameter Tuning*)

Kualitas Pareto-optimal yang dihasilkan oleh algoritma metaheuristik MOWaOA sangat dipengaruhi oleh kombinasi parameter kontrol internalnya. Oleh karena itu, dilakukan eksperimen penalaan parameter (*parameter tuning*) menggunakan metode *full factorial design* antara ukuran populasi walrus (N) dan jumlah iterasi maksimum (*Max Iterations*). Nilai variasi parameter yang diuji pada setiap berkas eksperimen dijabarkan pada Tabel 2.

Tabel 2: Variasi Nilai Parameter MOWaOA dalam Eksperimen

Parameter Kontrol	Nilai / Variasi yang Diuji
Ukuran Populasi Walrus (N)	25, 50, 100
Jumlah Iterasi Maksimum (<i>Max Iterations</i>)	50, 100, 150
Total Kombinasi Skenario	9 Kombinasi per Berkas Uji

Setiap skenario dari 9 kombinasi tersebut dijalankan secara independen pada masing-masing data uji (`tai100_5` hingga `tai100_20`) untuk menganalisis konvergensi parameter yang mampu menghasilkan solusi terbaik dengan waktu komputasi yang efisien.

6.3 Metrik Evaluasi Performa (Analisis Multicriterion Rajendran)

Dikarenakan terdapat dua buah objektif yang ingin diminimalkan (*Makespan* dan *Total Flow Time*), maka evaluasi performa dilakukan dengan mengadopsi prosedur penilaian *multicriterion* yang dikembangkan oleh Rajendran. Prosedur ini digunakan untuk membandingkan kualitas gabungan dua sekuens urutan penjadwalan, yaitu sekuens hasil Heuristik Rajendran standar (S) dan sekuens hasil optimasi algoritma MOWaOA (S').

Misalkan sekuens S memiliki nilai *Makespan* (MS) dan *Total Flow Time* (TF), sedangkan sekuens S' memiliki nilai *Makespan* (MS') dan *Total Flow Time* (TF'). Nilai indeks *multicriterion* R (untuk sekuens S) dan R' (untuk sekuens S') dihitung menggunakan Persamaan 13 dan Persamaan 14.

$$R = \frac{MS - \min(MS, MS')}{\min(MS, MS')} + \frac{TF - \min(TF, TF')}{\min(TF, TF')} \quad (13)$$

$$R' = \frac{MS' - \min(MS, MS')}{\min(MS, MS')} + \frac{TF' - \min(TF, TF')}{\min(TF, TF')} \quad (14)$$

Penentuan sekuens yang paling optimal didasarkan pada kriteria berikut:

- Jika $R' < R$, maka sekuens urutan penjadwalan S' (MOWaOA) dinyatakan lebih optimal dibandingkan sekuens S (Rajendran).
- Selisih atau perbedaan antara nilai R dan R' menjadi indikator kuantitatif seberapa baik kualitas gabungan nilai *Makespan* dan *Total Flow Time* yang berhasil ditingkatkan oleh algoritma usulan.

7 Hasil dan Analisis Eksperimen

Pengujian dilakukan pada lima berkas *benchmark* Taillard yaitu `tai50_10`, `tai50_20`, `tai100_5`, `tai100_10`, dan `tai100_20`. Solusi terbaik yang dihasilkan oleh algoritma MOWaOA (S') diadu secara langsung dengan solusi dari Heuristik Rajendran (S) menggunakan perhitungan *multi-criterion* Persamaan 13 dan 14. Hasil eksperimen disajikan sebagai berikut:

7.1 Benchmark tai50_5

7.1.1 Instance 1 (Lower Bound: 2712, Upper Bound: 2724)

Tabel 3: Hasil Eksperimen – `tai50_5`, Instance 1

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	2761,0	77.146,0
25	100	2752,0	76.392,0
25	150	2740,0	75.602,0
50	50	2739,0	77.511,0
50	100	2762,0	75.056,0
50	150	2745,0	77.506,0
100	50	2746,0	80.062,0
100	100	2741,0	74.963,0
100	150	2741,0	79.618,0

Best: populasi = 100, iterasi = 50, makespan = 2746,0

7.1.2 Instance 2 (Lower Bound: 2808, Upper Bound: 2834)

Tabel 4: Hasil Eksperimen – `tai50_5`, Instance 2

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	2928,0	79.594,0
25	100	2896,0	76.656,0
25	150	2879,0	80.816,0
50	50	2914,0	78.562,0
50	100	2884,0	77.341,0
50	150	2882,0	77.818,0
100	50	2882,0	78.518,0
100	100	2883,0	77.110,0
100	150	2860,0	78.283,0

Best: populasi = 25, iterasi = 150, makespan = 2879,0

7.1.3 Instance 3 (Lower Bound: 2596, Upper Bound: 2621)

Tabel 5: Hasil Eksperimen – tai50_5, Instance 3

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	2721,0	76.698,0
25	100	2689,0	77.998,0
25	150	2708,0	80.149,0
50	50	2727,0	79.346,0
50	100	2710,0	74.982,0
50	150	2709,0	74.985,0
100	50	2706,0	76.028,0
100	100	2714,0	77.212,0
100	150	2707,0	79.019,0

Best: populasi = 25, iterasi = 100, makespan = 2689,0

7.1.4 Instance 4 (Lower Bound: 2470, Upper Bound: 2751)

Tabel 6: Hasil Eksperimen – tai50_5, Instance 4

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	2821,0	82.867,0
25	100	2826,0	78.208,0
25	150	2804,0	79.618,0
50	50	2795,0	80.064,0
50	100	2826,0	76.639,0
50	150	2806,0	78.409,0
100	50	2790,0	81.722,0
100	100	2801,0	78.835,0
100	150	2707,0	82.497,0

Best: populasi = 100, iterasi = 150, makespan = 2707,0

7.1.5 Instance 5 (Lower Bound: 2837, Upper Bound: 2863)

Tabel 7: Hasil Eksperimen – tai50_5, Instance 5

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	2915,0	82.512,0
25	100	2881,0	79.778,0
25	150	2897,0	84.198,0
50	50	2922,0	77.906,0
50	100	2891,0	78.086,0
50	150	2878,0	79.469,0
100	50	2891,0	79.215,0
100	100	2882,0	79.391,0
100	150	2890,0	77.054,0

Best: populasi = 25, iterasi = 150, makespan = 2897,0

7.1.6 Instance 6 (Lower Bound: 2793, Upper Bound: 2829)

Tabel 8: Hasil Eksperimen – tai50_5, Instance 6

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	2935,0	81.728,0
25	100	2894,0	80.556,0
25	150	2887,0	80.891,0
50	50	2932,0	77.700,0
50	100	2878,0	77.766,0
50	150	2888,0	79.943,0
100	50	2875,0	78.551,0
100	100	2896,0	79.963,0
100	150	2870,0	79.962,0

Best: populasi = 25, iterasi = 150, makespan = 2887,0

7.1.7 Instance 7 (Lower Bound: 2689, Upper Bound: 2725)

Tabel 9: Hasil Eksperimen – tai50_5, Instance 7

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	2795,0	75392,0
25	100	2812,0	79.146,0
25	150	2798,0	74.332,0
50	50	2834,0	76.074,0
50	100	2788,0	77.562,0
50	150	2777,0	76.578,0
100	50	2829,0	75.320,0
100	100	2802,0	75.762 ,0
100	150	2766,0	73.995,0

Best: populasi = 25, iterasi = 100, makespan = 2812,0

7.1.8 Instance 8 (Lower Bound: 2667, Upper Bound: 2683)

Tabel 10: Hasil Eksperimen – tai50_5, Instance 8

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	2789,0	77.853,0
25	100	2763,0	74.993,0
25	150	2749,0	76.952,0
50	50	2745,0	75.891,0
50	100	2725,0	74.833,0
50	150	2720,0	73.890,0
100	50	2734,0	76.336,0
100	100	2711,0	72.852,0
100	150	2741,0	76.623,0

Best: populasi = 25, iterasi = 150, makespan = 2749,0

7.1.9 Instance 9 (Lower Bound: 2527, Upper Bound: 2552)

Tabel 11: Hasil Eksperimen – tai50_5, Instance 9

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	2639,0	75.759,0
25	100	2673,0	71.970,0
25	150	2656,0	74.601,0
50	50	2614,0	78.082,0
50	100	2622,0	75.001,0
50	150	2627,0	70.629,0
100	50	2607,0	73.395,0
100	100	2595,0	73.761,0
100	150	2597,0	72.433,0

Best: populasi = 50, iterasi = 50, makespan = 2614,0

7.1.10 Instance 10 (Lower Bound: 2776, Upper Bound: 2782)

Tabel 12: Hasil Eksperimen – tai50_5, Instance 10

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	2814,0	80.450,0
25	100	2810,0	82.263,0
25	150	2801,0	81.479,0
50	50	2836,0	80.371,0
50	100	2796,0	80.917,0
50	150	2799,0	82.245,0
100	50	2793,0	77.133,0
100	100	2792,0	79.587,0
100	150	2795,0	80.985,0

Best: populasi = 50, iterasi = 150, makespan = 2799,0

7.2 Benchmark tai50_10

7.2.1 Instance 1 (Lower Bound: 2907, Upper Bound: 3025)

Tabel 13: Hasil Eksperimen – tai50_10, Instance 1

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	3308,0	105.654,0
25	100	3270,0	103.797,0
25	150	3291,0	101.400,0
50	50	3302,0	101.073,0
50	100	3297,0	102.075,0
50	150	3333,0	100.578,0
100	50	3288,0	102.727,0
100	100	3259,0	101.622,0
100	150	3228,0	100.282,0

Best: populasi = 25, iterasi = 50, makespan = 3230,0

7.2.2 Instance 2 (Lower Bound: 3062, Upper Bound: 3107)

Tabel 14: Hasil Eksperimen – tai50_10, Instance 2

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	3349,0	102.270,0
25	100	3347,0	102.806,0
25	150	3352,0	103.489,0
50	50	3297,0	101.793,0
50	100	3347,0	99.285,0
50	150	3322,0	101.379,0
100	50	3313,0	100.972,0
100	100	3295,0	101.618,0
100	150	3324,0	98.160,0

Best: populasi = 50, iterasi = 50, makespan = 3295,0

7.2.3 Instance 3 (Lower Bound: 2959, Upper Bound: 3039)

Tabel 15: Hasil Eksperimen – tai50_10, Instance 3

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	3300,0	101.915,0
25	100	3331,0	100.948,0
25	150	3278,0	96.843,0
50	50	3240,0	97.312,0
50	100	3240,0	97.167,0
50	150	3230,0	97.541,0
100	50	3275,0	100.417,0
100	100	3296,0	99.249,0
100	150	3251,0	100.906,0

Best: populasi = 100, iterasi = 150, makespan = 3230,0

7.2.4 Instance 4 (Lower Bound: 2795, Upper Bound: 2902)

Tabel 16: Hasil Eksperimen – tai50_10, Instance 4

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	3259,0	98.760,0
25	100	3260,0	98.682,0
25	150	3172,0	100.659,0
50	50	3177,0	101.665,0
50	100	3163,0	96.183,0
50	150	3207,0	97.075,0
100	50	3160,0	98.627,0
100	100	3113,0	98.242,0
100	150	3159,0	97.462,0

Best: populasi = 50, iterasi = 50, makespan = 3113,0

7.2.5 Instance 5 (Lower Bound: 3046, Upper Bound: 3091)

Tabel 17: Hasil Eksperimen – tai50_10, Instance 5

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	3423,0	104.873,0
25	100	3363,0	102.034,0
25	150	3353,0	101.334,0
50	50	3371,0	101.630,0
50	100	3351,0	103.561,0
50	150	3338,0	97.505,0
100	50	3367,0	100.878,0
100	100	3316,0	100.089,0
100	150	3300,0	101.058,0

Best: populasi = 50, iterasi = 100, makespan = 3300,0

7.3 Benchmark tai100_5

7.3.1 Instance 1 (Lower Bound: 5208, Upper Bound: 5268)

Tabel 18: Hasil Eksperimen – tai100_5, Instance 1

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	5386,0	294.396,0
25	100	5333,0	282.098,0
25	150	5373,0	304.115,0
50	50	5358,0	287.890,0
50	100	5317,0	286.396,0
50	150	5331,0	292.255,0
100	50	5345,0	288.845,0
100	100	5316,0	288.361,0
100	150	5338,0	301.141,0

Best: populasi = 25, iterasi = 150, makespan = 5316,0

7.3.2 Instance 2 (Lower Bound: 5130, Upper Bound: 5175)

Tabel 19: Hasil Eksperimen – tai100_5, Instance 2

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	5305,0	284.708,0
25	100	5264,0	294.576,0
25	150	5271,0	281.496,0
50	50	5263,0	284.717,0
50	100	5289,0	289.812,0
50	150	5248,0	280.721,0
100	50	5272,0	278.041,0
100	100	5279,0	283.064,0
100	150	5293,0	285.720,0

Best: populasi = 25, iterasi = 100, makespan = 5248,0

7.3.3 Instance 3 (Lower Bound: 4963, Upper Bound: 5014)

Tabel 20: Hasil Eksperimen – tai100_5, Instance 3

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	5127,0	268.777,0
25	100	5061,0	268.459,0
25	150	5072,0	271.898,0
50	50	5095,0	282.150,0
50	100	5088,0	278.516,0
50	150	5055,0	268.485,0
100	50	5111,0	273.175,0
100	100	5044,0	266.813,0
100	150	5093,0	278.043,0

Best: populasi = 50, iterasi = 50, makespan = 5044,0

7.3.4 Instance 4 (Lower Bound: 5195, Upper Bound: 5250)

Tabel 21: Hasil Eksperimen – tai100_5, Instance 4

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	5402,0	289.422,0
25	100	5357,0	281.031,0
25	150	5352,0	295.031,0
50	50	5398,0	291.151,0
50	100	5367,0	286.712,0
50	150	5342,0	282.066,0
100	50	5314,0	275.326,0
100	100	5328,0	288.517,0
100	150	5321,0	287.882,0

Best: populasi = 25, iterasi = 150, makespan = 5314,0

7.3.5 Instance 5 (Lower Bound: 5063, Upper Bound: 5135)

Tabel 22: Hasil Eksperimen – tai100_5, Instance 5

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	5244,0	279.234,0
25	100	5227,0	280.879,0
25	150	5218,0	284.812,0
50	50	5202,0	284.706,0
50	100	5197,0	284.712,0
50	150	5175,0	273.894,0
100	50	5184,0	282.404,0
100	100	5188,0	282.660,0
100	150	5161,0	272.552,0

Best: populasi = 50, iterasi = 100, makespan = 5161,0

7.3.6 Instance 6 (Lower Bound: 5198, Upper Bound: 5246)

Tabel 23: Hasil Eksperimen – tai100_5, Instance 6

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	5403,0	288.416,0
25	100	5343,0	282.696,0
25	150	5326,0	292.013,0
50	50	5361,0	289.936,0
50	100	5335,0	286.804,0
50	150	5305,0	281.097,0
100	50	5344,0	280.517,0
100	100	5319,0	288.153,0
100	150	5313,0	287.799,0

Best: populasi = 25, iterasi = 150, makespan = 5305,0

7.3.7 Instance 7 (Lower Bound: 5038, Upper Bound: 5106)

Tabel 24: Hasil Eksperimen – tai100_5, Instance 7

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	5288,0	296.120,0
25	100	5235,0	275.780,0
25	150	5173,0	273.649,0
50	50	5197,0	276.455,0
50	100	5184,0	278.913,0
50	150	5163,0	279.787,0
100	50	5187,0	285.173,0
100	100	5170,0	289.318,0
100	150	5169,0	281.965,0

Best: populasi = 25, iterasi = 50, makespan = 5163,0

7.3.8 Instance 8 (Lower Bound: 5385, Upper Bound: 5454)

Tabel 25: Hasil Eksperimen – tai100_5, Instance 8

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	5560,0	301.933,0
25	100	5566,0	299.793,0
25	150	5521,0	292.572,0
50	50	5533,0	302.048,0
50	100	5575,0	297.243,0
50	150	5514,0	294.802,0
100	50	5514,0	298.675,0
100	100	5554,0	301.039,0
100	150	5526,0	299.250,0

Best: populasi = 50, iterasi = 50, makespan = 5514,0

7.3.9 Instance 9 (Lower Bound: 5272, Upper Bound: 5328)

Tabel 26: Hasil Eksperimen – tai100_5, Instance 9

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	5454,0	305.157,0
25	100	5450,0	288.631,0
25	150	5393,0	281.011,0
50	50	5451,0	294.699,0
50	100	5431,0	293.163,0
50	150	5428,0	297.764,0
100	50	5386,0	295.730,0
100	100	5386,0	289.381,0
100	150	5414,0	295.073,0

Best: populasi = 25, iterasi = 50, makespan = 5386,0

7.3.10 Instance 10 (Lower Bound: 5148, Upper Bound: 5200)

Tabel 27: Hasil Eksperimen – tai100_5, Instance 10

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	5554,0	297.224,0
25	100	5554,0	297.224,0
25	150	5538,0	299.010,0
50	50	5586,0	303.856,0
50	100	5569,0	294.501,0
50	150	5540,0	303.142,0
100	50	5548,0	303.632,0
100	100	5564,0	294.105,0
100	150	5538,0	293.440,0

Best: populasi = 100, iterasi = 50, makespan = 5538,0

7.4 Benchmark tai100_10

7.4.1 Instance 1 (Lower Bound: 5759, Upper Bound: 5770)

Tabel 28: Hasil Eksperimen – tai100_10, Instance 1

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	6276,0	362.354,0
25	100	6206,0	346.553,0
25	150	6261,0	367.675,0
50	50	6179,0	361.573,0
50	100	6209,0	354.023,0
50	150	6148,0	346.545,0
100	50	6125,0	353.454,0
100	100	6109,0	351.890,0
100	150	6130,0	344.384,0

Best: populasi = 25, iterasi = 150, makespan = 6109,0

7.4.2 Instance 2 (Lower Bound: 5345, Upper Bound: 5349)

Tabel 29: Hasil Eksperimen – tai100_10, Instance 2

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	5763,0	340.240,0
25	100	5878,0	330.769,0
25	150	5796,0	333.990,0
50	50	5811,0	330.990,0
50	100	5788,0	326.113,0
50	150	5773,0	324.364,0
100	50	5664,0	327.883,0
100	100	5753,0	329.380,0
100	150	5696,0	322.302,0

Best: populasi = 25, iterasi = 50, makespan = 5664,0

7.4.3 Instance 3 (Lower Bound: 5623, Upper Bound: 5677)

Tabel 30: Hasil Eksperimen – tai100_10, Instance 3

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	6115,0	348.116,0
25	100	5971,0	341.781,0
25	150	5997,0	336.364,0
50	50	6024,0	346.760,0
50	100	6039,0	351.354,0
50	150	5952,0	344.491,0
100	50	6027,0	339.281,0
100	100	6030,0	349.845,0
100	150	5913,0	341.395,0

Best: populasi = 50, iterasi = 100, makespan = 5913,0

7.4.4 Instance 4 (Lower Bound: 5732, Upper Bound: 5791)

Tabel 31: Hasil Eksperimen – tai100_10, Instance 4

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	6243,0	357.119,0
25	100	6234,0	355.590,0
25	150	6261,0	356.205,0
50	50	6260,0	365.059,0
50	100	6272,0	359.552,0
50	150	6255,0	357.378,0
100	50	6265,0	364.720,0
100	100	6257,0	351.142,0
100	150	6147,0	355.072,0

Best: populasi = 50, iterasi = 50, makespan = 6147,0

7.4.5 Instance 5 (Lower Bound: 5431, Upper Bound: 5468)

Tabel 32: Hasil Eksperimen – tai100_10, Instance 5

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	5949,0	332.278,0
25	100	5958,0	338.869,0
25	150	6042,0	348.748,0
50	50	5870,0	339.599,0
50	100	5948,0	339.961,0
50	150	5956,0	337.402,0
100	50	5933,0	340.547,0
100	100	5894,0	339.390,0
100	150	5862,0	339.431,0

Best: populasi = 50, iterasi = 50, makespan = 5862,0

7.4.6 Instance 6 (Lower Bound: 5246, Upper Bound: 5303)

Tabel 33: Hasil Eksperimen – tai100_10, Instance 6

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	5734,0	331.717,0
25	100	5706,0	317.035,0
25	150	5605,0	325.347,0
50	50	5691,0	329.976,0
50	100	5670,0	330.496,0
50	150	5632,0	326.470,0
100	50	5666,0	331.615,0
100	100	5626,0	324.436,0
100	150	5624,0	324.853,0

Best: populasi = 100, iterasi = 50, makespan = 5666,0

7.4.7 Instance 7 (Lower Bound: 5523, Upper Bound: 5599)

Tabel 34: Hasil Eksperimen – tai100_10, Instance 7

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	5954,0	335.121,0
25	100	5945,0	333.919,0
25	150	6003,0	342.501,0
50	50	5936,0	340.550,0
50	100	5924,0	338.174,0
50	150	5870,0	334.570,0
100	50	5853,0	337.093,0
100	100	5897,0	334.674,0
100	150	5898,0	329.467,0

Best: populasi = 100, iterasi = 50, makespan = 5853,0

7.4.8 Instance 8 (Lower Bound: 5556, Upper Bound: 5623)

Tabel 35: Hasil Eksperimen – tai100_10, Instance 8

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	6013,0	345.586,0
25	100	6032,0	338.381,0
25	150	5966,0	337.792,0
50	50	6009,0	340.244,0
50	100	6007,0	350.086,0
50	150	6008,0	346.713,0
100	50	6026,0	346.453,0
100	100	5921,0	335.984,0
100	150	5945,0	343.994,0

Best: populasi = 50, iterasi = 100, makespan = 6007,0

7.4.9 Instance 9 (Lower Bound: 5779, Upper Bound: 5875)

Tabel 36: Hasil Eksperimen – tai100_10, Instance 9

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	6260,0	358.202,0
25	100	6253,0	356.713,0
25	150	6207,0	359.599,0
50	50	6185,0	354.606,0
50	100	6173,0	357.249,0
50	150	6154,0	349.491,0
100	50	6228,0	356.200,0
100	100	6176,0	348.645,0
100	150	6120,0	351.316,0

Best: populasi = 25, iterasi = 150, makespan = 6207,0

7.4.10 Instance 10 (Lower Bound: 5830, Upper Bound: 5845)

Tabel 37: Hasil Eksperimen – tai100_10, Instance 10

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	6202,0	351.765,0
25	100	6114,0	347.352,0
25	150	6214,0	347.971,0
50	50	6148,0	352.032,0
50	100	6163,0	347.760,0
50	150	6142,0	348.519,0
100	50	6123,0	345.173,0
100	100	6184,0	343.329,0
100	150	6009,0	335.772,0

Best: populasi = 50, iterasi = 50, makespan = 6148,0

7.5 Benchmark tai100_20

7.5.1 Instance 1 (Lower Bound: 5851, Upper Bound: 6286)

Tabel 38: Hasil Eksperimen – tai100_20, Instance 1

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	7137,0	429.599,0
25	100	7108,0	435.822,0
25	150	7092,0	433.691,0
50	50	7103,0	441.531,0
50	100	7068,0	430.287,0
50	150	7079,0	434.788,0
100	50	7009,0	426.726,0
100	100	6991,0	416.665,0
100	150	6996,0	420.069,0

Best: populasi = 50, iterasi = 100, makespan = 7068,0

7.5.2 Instance 2 (Lower Bound: 6099, Upper Bound: 6241)

Tabel 39: Hasil Eksperimen – tai100_20, Instance 2

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	7256,0	441.051,0
25	100	7091,0	435.074,0
25	150	7064,0	432.587,0
50	50	7162,0	441.506,0
50	100	7024,0	431.951,0
50	150	7009,0	439.142,0
100	50	7068,0	429.929,0
100	100	6930,0	430.947,0
100	150	6987,0	427.956,0

Best: populasi = 50, iterasi = 150, makespan = 7009,0

7.5.3 Instance 3 (Lower Bound: 6099, Upper Bound: 6329)

Tabel 40: Hasil Eksperimen – tai100_20, Instance 3

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	7119,0	435.811,0
25	100	7149,0	433.016,0
25	150	7067,0	425.064,0
50	50	7010,0	431.411,0
50	100	7065,0	430.562,0
50	150	7028,0	424.765,0
100	50	7053,0	434.367,0
100	100	7028,0	423.062,0
100	150	7100,0	429.762,0

Best: populasi = 100, iterasi = 50, makespan = 7053,0

7.5.4 Instance 4 (Lower Bound: 6072, Upper Bound: 6306)

Tabel 41: Hasil Eksperimen – tai100_20, Instance 4

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	7088,0	431.999,0
25	100	6992,0	430.644,0
25	150	7075,0	432.718,0
50	50	7056,0	441.230,0
50	100	7140,0	435.204,0
50	150	6971,0	430.632,0
100	50	7057,0	438.228,0
100	100	6973,0	425.633,0
100	150	7025,0	431.168,0

Best: populasi = 50, iterasi = 150, makespan = 6971,0

7.5.5 Instance 5 (Lower Bound: 6009, Upper Bound: 6377)

Tabel 42: Hasil Eksperimen – tai100_20, Instance 5

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	7219,0	437.665,0
25	100	7126,0	447.187,0
25	150	7145,0	434.700,0
50	50	7100,0	438.158,0
50	100	7174,0	432.158,0
50	150	7031,0	440.109,0
100	50	7049,0	436.099,0
100	100	7051,0	429.235,0
100	150	7129,0	428.373,0

Best: populasi = 25, iterasi = 100, makespan = 7126,0

7.5.6 Instance 6 (Lower Bound: 6144, Upper Bound: 6437)

Tabel 43: Hasil Eksperimen – tai100_20, Instance 6

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	7196,0	437.604,0
25	100	7120,0	430.136,0
25	150	7100,0	438.353,0
50	50	7186,0	433.748,0
50	100	7149,0	438.392,0
50	150	7220,0	433.712,0
100	50	7132,0	436.104,0
100	100	7085,0	440.408,0
100	150	7177,0	435.675,0

Best: populasi = 100, iterasi = 100, makespan = 7085,0

7.5.7 Instance 7 (Lower Bound: 5991, Upper Bound: 6346)

Tabel 44: Hasil Eksperimen – tai100_20, Instance 7

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	7262,0	441.176,0
25	100	7172,0	444.578,0
25	150	7122,0	433.659,0
50	50	7234,0	441.691,0
50	100	7171,0	433.964,0
50	150	7178,0	436.993,0
100	50	7163,0	438.554,0
100	100	7094,0	432.375,0
100	150	7080,0	430.398,0

Best: populasi = 25, iterasi = 100, makespan = 7172,0

7.5.8 Instance 8 (Lower Bound: 6084, Upper Bound: 6481)

Tabel 45: Hasil Eksperimen – tai100_20, Instance 8

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	7330,0	458.609,0
25	100	7351,0	446.620,0
25	150	7290,0	442.926,0
50	50	7281,0	441.824,0
50	100	7179,0	446.260,0
50	150	7227,0	445.159,0
100	50	7162,0	448.853,0
100	100	7236,0	450.041,0
100	150	7150,0	438.802,0

Best: populasi = 100, iterasi = 50, makespan = 7162,0

7.5.9 Instance 9 (Lower Bound: 5979, Upper Bound: 6358)

Tabel 46: Hasil Eksperimen – tai100_20, Instance 9

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	7177,0	446.114,0
25	100	7176,0	430.484,0
25	150	7098,0	440.023,0
50	50	7226,0	443.044,0
50	100	7120,0	437.474,0
50	150	7026,0	428.230,0
100	50	7156,0	438.055,0
100	100	7038,0	438.656,0
100	150	7071,0	427.946,0

Best: populasi = 50, iterasi = 150, makespan = 7026,0

7.5.10 Instance 10 (Lower Bound: 6298, Upper Bound: 6465)

Tabel 47: Hasil Eksperimen – tai100_20, Instance 10

Pop. Size	Max Iter	Makespan	Total Flow Time
25	50	7277,0	445.633,0
25	100	7217,0	439.056,0
25	150	7193,0	438.353,0
50	50	7255,0	446.893,0
50	100	7160,0	435.934,0
50	150	7168,0	434.928,0
100	50	7213,0	448.082,0
100	100	7099,0	444.912,0
100	150	7113,0	442.940,0

Best: populasi = 100, iterasi = 100, makespan = 7099,0

7.6 Analisis Eksperimen

Berdasarkan hasil eksperimen, ditemukan pola unik pada performa algoritma terkait kombinasi parameter. Temuan menunjukkan bahwa konfigurasi Populasi 25 dengan Iterasi 150 merupakan kombinasi yang paling sering menghasilkan nilai objektif optimal. Pola ini mengindikasikan bahwa algoritma MOWaOA memiliki karakteristik exploitation yang kuat. Dengan populasi kecil (25), algoritma mampu melakukan konvergensi yang cepat menuju nilai optimal. Penambahan jumlah iterasi (hingga 150) memberikan waktu komputasi yang cukup bagi algoritma untuk menyempurnakan (fine-tuning) solusi yang sudah ditemukan, tanpa memerlukan memori yang besar untuk populasi yang banyak.

Di sisi lain, meskipun populasi 25 efisien, pada masalah dengan dimensi yang lebih kompleks (seperti tai100_5), penggunaan populasi yang lebih besar (50 atau 100) tetap diperlukan. Hal ini bertujuan untuk menjaga keragaman (diversity) solusi dan menghindari terjebak pada local optima. Dengan demikian, MOWaOA menunjukkan perilaku yang adaptif tergantung pada kompleksitas instans masalah.

8 Kesimpulan

Algoritma Multi-Objective Walrus Optimization Algorithm (MOWaOA) memiliki tingkat konvergensi dan efektivitas yang tinggi dalam menyelesaikan masalah penjadwalan flow shop. Berdasarkan hasil eksperimen yang komprehensif, ditemukan bahwa kombinasi populasi 25 dengan 150 iterasi merupakan konfigurasi yang paling efisien, di mana algoritma mampu mencapai nilai optimal dengan waktu komputasi yang minimal. Hal ini menunjukkan bahwa MOWaOA memiliki kemampuan eksploitasi yang kuat, sehingga tidak selalu memerlukan ukuran populasi yang besar untuk menemukan solusi yang mendekati batas teoritis Lower Bound dan Upper Bound yang ditetapkan pada dataset benchmark Taillard. Ketangguhan algoritma juga teruji pada berbagai skala masalah, mulai dari 50 hingga 100 pekerjaan, di mana nilai makespan yang dihasilkan konsisten mendekati batas atas yang ada.

Secara keseluruhan, penerapan MOWaOA memberikan kontribusi praktis dalam mengoptimalkan sistem penjadwalan produksi dibandingkan dengan metode heuristik konvensional. Keberhasilan ini menunjukkan bahwa pendekatan metaheuristik berbasis perilaku Walrus mampu memberikan solusi penjadwalan yang lebih produktif dan efisien. Penelitian ini juga membuka jalan bagi pengembangan di masa depan, seperti integrasi dengan teknik pencarian lokal (local

search) untuk menangani masalah dengan dimensi yang lebih masif, atau penambahan variabel kendala produksi yang lebih kompleks guna mendekati kondisi rantai produksi yang sebenarnya.

9 Daftar Pustaka

1. Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. A. M. T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2), 182-197.
2. Taillard, E. (1993). Benchmarks for basic scheduling problems. *European Journal of Operational Research*, 64(2), 278-285.
3. Trojovský, P., & Dehghani, M. (2022). Walrus Optimization Algorithm: A New Bio-Inspired Metaheuristic Algorithm. *Research Square Preprint*. DOI: <https://doi.org/10.21203/rs.3.rs-2174098/v1>.
4. Pinedo, M. L. (2016). *Scheduling: Theory, Algorithms, and Systems* (5th ed.). Springer International Publishing.
5. Rajendran, C. (1994). A heuristic for scheduling in flowshop to minimize makespan and total flowtime. *International Journal of Production Economics*, 33(1-3), 121-134.
6. Talbi, E. G. (2009). *Metaheuristics: From Design to Implementation*. John Wiley & Sons.
7. Wang, G. G., et al. (2023). Walrus Optimization Algorithm: A new nature-inspired metaheuristic algorithm. *Expert Systems with Applications*, 227, 120-145.
8. Zadeh, P. M., et al. (2022). Multi-objective optimization of flow shop scheduling problems using metaheuristics. *Journal of Manufacturing Systems*, 63, 450-468.